WFLOW-02: Triggers & Event Types

> **Series:** WFLOW — Workflows and Alert Notifications | **Notebook:** 2 of 10 | **Created:** January 2026 | **Last Updated:** 11/11/2026

Event-Driven Workflow Triggers

Triggers determine when workflows execute. This notebook covers all trigger types, detected problem events, Davis events, schedules, and custom event triggers.

Table of Contents

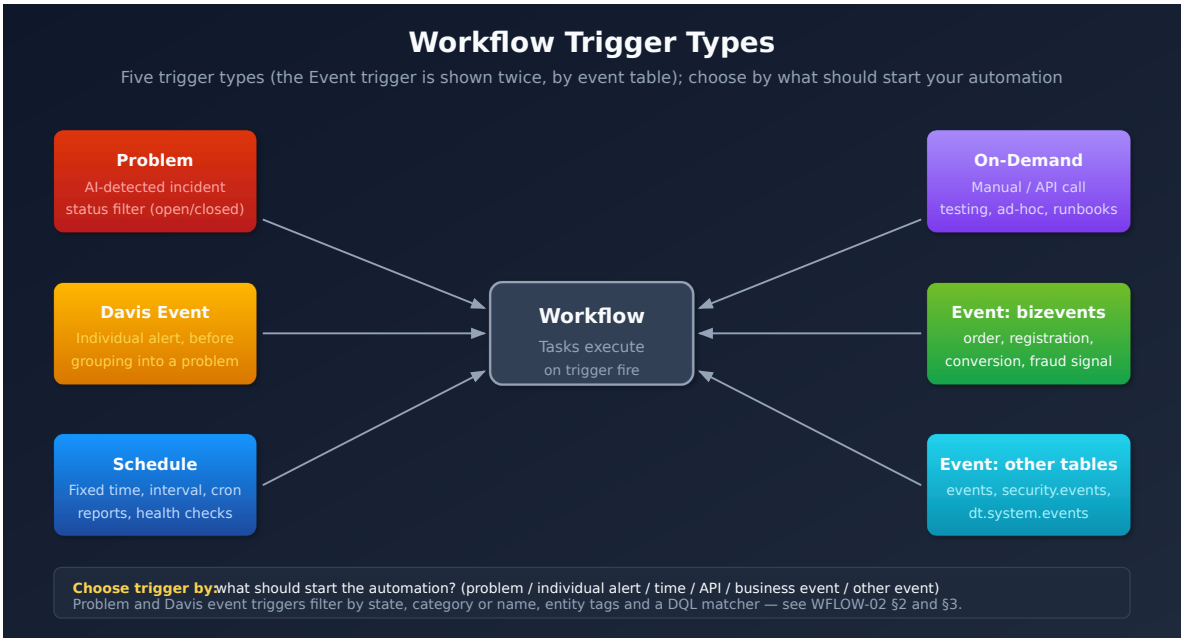
1. Trigger Types Overview
2. Detected Problem Trigger
3. Davis Event Trigger
4. Schedule Trigger
5. On-Demand Trigger
6. Event Trigger (Custom/Business Events)
7. Trigger Data and Expressions
8. Davis Problem Event Payload Reference

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription
Permissions	`automation:workflows:write` (and `automation:workflows:run` for API runs, §5). The workflow's **actor** also needs read access to the table each trigger watches (§1)
Prior Knowledge	**WFLOW-01: Workflow Fundamentals**

1. Trigger Types Overview

Trigger Type	Fires When	Primary Use Case
Detected Problem	Dynatrace Intelligence detects/updates/closes a problem	Alert notifications, incident management
Davis Event	An anomaly detector raises a Davis event	Per-alert automation, before problem grouping
Schedule	A fixed time, a time interval or a cron expression comes due	Reports, health checks, cleanup jobs
On-Demand	Manual execution or API call	Testing, ad-hoc automation
Event	Any event matching a DQL matcher in `events`, `bizevents`, `security.events` or `dt.system.events`	Business process automation



Two reasons a trigger never fires. First, *"Triggers only start on live workflows."* A saved draft runs only from the **Run** button; select **Deploy** before you wait for a problem, event or schedule. Second, the actor must be able to read what the trigger watches: *"The workflow actor requires read access to the event type the trigger reacts on."*

Trigger	Actor permission
Problem, Davis event, Event (`events`)	`storage:events:read`
Event (`bizevents`)	`storage:bizevents:read`
Event (`security.events`)	`storage:security.events:read`

Trigger	Actor permission
Event (`dt.system.events`)	`storage:system:read`

Add `storage:buckets:read`, restricted to the bucket that holds the events. Use a service user as the actor for production workflows (WFLOW-01 §3.5).

Sources: Workflow triggers (DT docs), Event triggers for workflows (DT docs).

Choosing the Right Trigger

Scenario	Recommended Trigger
"Notify when a service is slow"	Detected Problem
"Alert when CPU > 90% for 5 mins"	An anomaly detector for the threshold (ALERT-02), then Detected Problem
"Send weekly status report"	Schedule
"Process order completion events"	Event (bizevents)
"Test my workflow"	On-Demand

2. Detected Problem Trigger

The most common trigger for alert notifications. Fires when Dynatrace Intelligence detects problems.

Trigger Configuration

Setting	Description	Example
Problem state	`active` (default) starts when the problem opens; `active or closed` also when it closes; `closed` only when it closes	`active`
Event category	Problem categories to include	Availability, Error, Slowdown
Severity	The level at or above which problems start the workflow (see the SaaS 1.348 note in WFLOW-04 §3 before relying on it)	Any

Setting	Description	Example
Affected entities — tags	Tags the problem's affected entities must carry. Three modes: include all / all defined tags / any defined tag	`team:checkout`
Additional custom filter query	A DQL **matcher** on the incoming problem record (a subset of DQL — no aggregation, no querying across a set of events)	`maintenance.is_under_maintenance == false`
Minimum duration (Advanced options)	Postpones the trigger until the problem has been open this long: 5, 10, 15, 30, 60, 120, 240, 1440 or 10080 minutes	`15`
Updates (Advanced options)	Re-trigger when selected fields change. Without it, the trigger starts once per state transition	Root cause changed
Wait for root cause analysis (Advanced options)	Start only after root cause analysis has completed for the problem	On

Enable **Wait for root cause analysis** unless you need the earliest possible signal. The reference says *"Recommended: Enable this to avoid triggering on incomplete problem data."* All of these fields compile into one DQL matcher, which is limited to 1,000 characters, so every selected category and tag uses part of that budget.

> ⚠️ **There is no Management Zone filter on the problem trigger.** An earlier revision of this notebook listed one; that was wrong. Dynatrace's upgrade guide describes the replacement directly: *"A workflow's Problem trigger filters problems directly with DQL matchers on the problem. There is no separate filter object to create, name, and maintain, nor is there a one-management-zone-per-profile constraint."* The trigger's configuration (problem state, event category, severity, affected-entity tags, minimum duration, updates, wait for root cause analysis, custom filter) has no Management Zone option (Event triggers for workflows (DT docs)). Scope the trigger with **affected-entity tags** plus the custom DQL matcher instead.
>
> This matters most if you are migrating off Management Zones: do not route on `event()["management_zones"]`. The field is **not present on problem records** — on a live tenant (10/05/2026) 0 of 15,878 problems in `dt.davis.problems` over 30 days carried it, while 14 of 15 hosts belonged to a management zone — so a condition that reads it has nothing to match, whether or not your zones still exist. Check your own tenant with `fetch dt.davis.problems, from:-30d | summarize n = countIf(isNotNull(management_zones))`. See MZ2POL-01 §5 for the full MZ-job-to-successor mapping.
>
> **Validate the filter before you save it.** The trigger configuration offers **Query past events**, which estimates how many matching events occurred in your environment over recent windows. Use it

on every non-trivial trigger.

The result to distrust is **zero across every window in an environment you know is busy**. A trigger can be syntactically valid, save without error, and still never fire — nothing surfaces an error to tell you, because an over-constrained filter is not a malformed one. Check in particular that the **categories selected above and any category named in the custom filter query agree**: a matcher narrowing to a category you did not select leaves nothing that can ever match. Reading the estimate at authoring time costs seconds and is the only signal you get.

The maintenance flag can be null on Davis events. Dynatrace documents the filter two ways — `maintenance.is_under_maintenance == false` in the upgrade guide and `maintenance.is_under_maintenance != true` on the maintenance-window data page — and in DQL both drop a record whose flag is null. On problems that did not matter on a validation tenant (10/14/2026): over 30 days, all 17,573 records in `dt.davis.problems` carried the flag (6,260 true, 11,313 false, 0 null). On Davis events it did: 15,411 of 993,339 records in `dt.davis.events` (about 1.6%) had no value, and `== false` and `!= true` each matched 960,808 — neither kept them. `coalesce(maintenance.is_under_maintenance, false) == false` matched 976,219, keeping them.

Use the `coalesce` form in DQL queries that report on Davis events. For a **trigger** matcher, how `==` and `!=` treat a null field is not documented, and neither is `coalesce`: the DQL matcher reference lists `isNull` (*"Tests if a value is NULL."*) but not `coalesce`. So this notebook makes no claim either way for the trigger — if null-flag Davis events matter to a Davis event trigger, test the matcher with **Query past events** before relying on it.

Sources: Upgrade from Classic maintenance windows to Latest maintenance windows (DT docs) — *"add maintenance.is_under_maintenance == false as a filter to exclude problems that occur during a maintenance window."*; Use maintenance window data in Grail (DT docs) — the `!= true` form; DQL matcher in OpenPipeline (DT docs) — *"Tests if a value is NULL."* Tenant counts: MCP `execute-dql`, 30 days to 10/14/2026, empty `notifications` arrays; the `dt.davis.problems` run is the control.

Problem Event Data

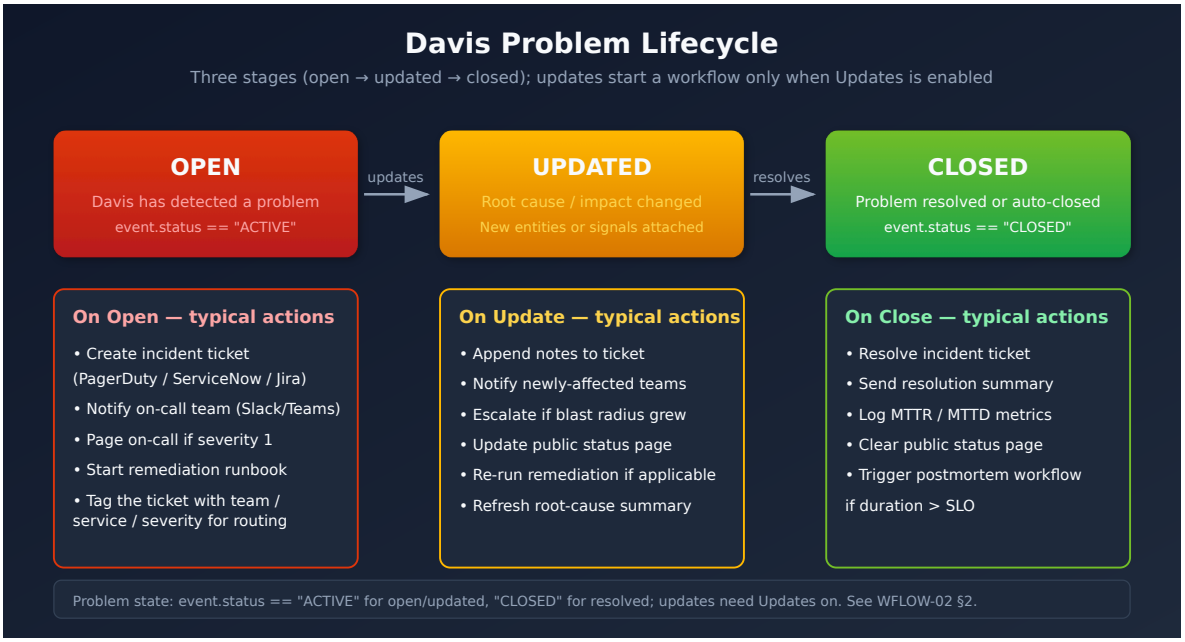
The payload is the `dt.davis.problems` record. Run `fetch dt.davis.problems, from:-24h | limit 1` to see every field your templates can read. An abridged record (field names as returned by a live tenant, 09/24/2026):

```
{
  "display_id": "P-260911127",
  "event.id": "-1001992832270454690_1790253360000V2",
  "event.name": "High response time on checkout service",
  "event.category": "SLOWDOWN",
  "event.status": "ACTIVE",
  "event.status_transition": "CREATED",
  "event.severity": 3,
  "event.start": "2026-09-24T12:37:00.000000000Z",
  "affected_entity_ids": ["SERVICE-ABC123"],
  "smartscape.affected_entities": [{"id": "SERVICE-ABC123", "name": "checkout", "type":
"SERVICE"}],
  "entity_tags": ["team:checkout"],
  "maintenance.is_under_maintenance": false
}
```


There is no `title`, `status`, `problem_url` or `start_time` field. The link to the problem comes from the `{{ problem_link() }}` expression, not from the record. §7 has the full mapping.

Problem Lifecycle Events

Event	When	Typical Action
Problem opened	New problem detected	Create incident, notify team
Problem updated (only with **Updates** enabled)	A tracked field changed (root cause, impact, affected entities, …)	Update incident notes
Problem closed	Problem resolved	Close incident, send summary



Example: Filter Critical Production Problems

This is the shape the trigger takes in an exported workflow. Values are illustrative: build the trigger in the editor and export the workflow for the exact document, which also carries the compiled `filterQuery`.

```
trigger:
  eventTrigger:
    isActive: true
    triggerConfiguration:
      type: davis-problem
      value:
        triggerOn: open              # open | open-and-close | close
        categories:                  # availability, error, slowdown, resource, custom,
          availability: true         # monitoringUnavailable, info — there is no
"performance"
          slowdown: true
        entityTagsMatch: all
        entityTags:
          env:
            - prod
        analysisReady: true          # Wait for root cause analysis
        problemOpenDuration: 15      # Minimum duration, in minutes
        customFilter: maintenance.is_under_maintenance == false
```

> **Sources:** Event triggers for workflows (DT docs) — *"Wait for root cause analysis : When enabled, the trigger starts only after Dynatrace Intelligence has completed root cause analysis for the problem."*; Automation client SDK (Dynatrace Developer) — `DavisProblemConfig` and `DavisProblemCategories` field names; wftpl_sample_servicenow_incident_man.yaml (Dynatrace GitHub) — an exported `davis-problem` trigger.

3. Davis Event Trigger

React to an individual Davis event, the alert an anomaly detector raises, rather than to the problem Dynatrace groups those alerts into.

> **This trigger does not evaluate a threshold.** Dynatrace documents it as *"Davis event trigger — Starts on individual alerts when anomalies are detected."* The threshold itself lives in an anomaly detector (**AIOPS-02**, **ALERT-02**); the trigger reacts to the Davis event that detector raises. An earlier revision of this notebook showed a trigger with its own DQL query and evaluation frequency. That trigger does not exist.

When to Use

The upgrade guide's advice: *"Trigger on problems, not on Davis events. Dynatrace Intelligence groups related alerts into one problem. Triggering on raw events bypasses that grouping and floods the channel. Use the Davis event trigger only when you genuinely need per-alert granularity."* Typical cases:

- Automation on non-problem events, such as deployments, configuration changes and informational signals
- A per-alert action where the grouped problem is too coarse

Configuration

Setting	Description
Problem state	active, active or closed, or closed
Davis event name	equals / contains a string
Affected entities	Filter by entity tags (all entities / all defined tags / any defined tag)
Maintenance window	Always, inside maintenance window only, or outside maintenance window only
Additional custom filter query	A DQL matcher over the Davis event record

Event Data

The payload is the `dt.davis.events` record. Run `fetch dt.davis.events, from:-24h | limit 1` to see its fields. Commonly used: `event.name`, `event.category`, `event.status`, `event.start`, `dt.source_entity`, `dt.smartscape_source.id`, `entity_tags`.

> **Sources:** Event triggers for workflows (DT docs), Upgrade guide — alerting and notifications (DT docs).

4. Schedule Trigger

Run workflows at a set time or interval. There are three schedule types:

Type	Use
Fixed time	A time of day (HH:MM) on the days a scheduling rule selects: every day, weekdays, or business days from a business calendar. Time zone defaults to UTC

Type	Use
Time interval	Every N minutes (1–720), optionally within active hours and a scheduling rule
Cron schedule	Standard five-field cron, for patterns the other two cannot express

Cron expressions use the Debian cron format. *"Six-field (Quartz) expressions and the ? wildcard are not supported."*

Cron Expression Format

```
┌──────────── minute (0-59)
│ ┌────────── hour (0-23)
│ │ ┌──────── day of month (1-31)
│ │ │ ┌────── month (1-12)
│ │ │ │ ┌──── day of week (0-6, Sun=0)
│ │ │ │ │
* * * * *
```

Common Schedules

Schedule	Cron Expression	Description
Every hour	`0 * * * *`	Top of every hour
Daily at 9 AM	`0 9 * * *`	Every day at 9:00
Weekdays at 8 AM	`0 8 * * 1-5`	Mon-Fri at 8:00
First of month	`0 0 1 * *`	1st day, midnight
Every 15 minutes	`*/15 * * * *`	0, 15, 30, 45 past

Example: Daily Health Check Report

The shape of a cron Schedule trigger in an exported workflow (illustrative; export a workflow built in the editor for the exact document):

```
trigger:
  schedule:
    isActive: true
    trigger:
      type: cron
      cron: "3 9 * * 1-5"      # Weekdays at 09:03, staggered off the hour
    timezone: America/New_York
```

Schedule Best Practices

- **Stagger start times.** Dynatrace's guidance: *"stagger their start times by a few minutes, for example, 00:00 , 00:03 , and 00:07 ."* Pinning many workflows to :00 bursts load on the systems they call.
- **Set the time zone explicitly.** For cron, *"The time zone defaults to your browser's locale."*
- **Check the next runs.** Use **Next scheduled executions** in the trigger configuration before saving.
- **Don't schedule more often than the work needs.** Each run costs Standard Function Calls, and DQL tasks consume query volume.

> **Sources:** Workflow schedule trigger (DT docs); Automation client SDK (Dynatrace Developer) — `Schedule` / `CronTrigger`.

5. On-Demand Trigger

Manual execution for testing, ad-hoc runs, or API-triggered automation.

Manual Execution

1. Open workflow in editor
2. Click **Run** button
3. Optionally provide input parameters
4. View execution results

API Execution

Trigger workflows programmatically via API:

```
curl -X POST "https://&lt;env&gt;/platform/automation/v1/workflows/&lt;id&gt;/run" \
  -H "Authorization: Bearer &lt;platform-token&gt;" \
  -H "Content-Type: application/json" \
  -d '{"input": {"environment": "prod", "notify_slack": true}}'
```

The platform token needs the `automation:workflows:run` scope. The call runs the **live** workflow: *"An API call can always trigger a live workflow, regardless of its trigger type."* A draft runs only from the editor: *"A workflow in draft mode can only be run via Workflows using the Run button."* Platform tokens use `Bearer`, not `Api-Token`: *"To use a platform token please provide the token in the Authorization header: Authorization: Bearer "* (Platform tokens (DT docs)).

Workflow Inputs

Values passed as `input` arrive as workflow inputs. *"The input is a merge of the default workflow inputs and the inputs provided at runtime."* Read them in tasks with `input()`:

```
{{ input('environment') }}
{{ input('notify_slack') }}
```

> **Sources:** Monitor workflow executions (DT docs) — the run endpoint's `input` and `params` attributes, Jinja expressions for Workflows (DT docs) — `input()`, Workflow triggers (DT docs), Automation Workflow consumption (DT docs).

6. Event Trigger (Custom/Business Events)

Trigger on business events or custom events ingested into Grail.

Use Cases

- **Order completed** → Update inventory system
- **User registered** → Send welcome notification
- **Deployment finished** → Run validation tests
- **Custom alert** → External system integration

Configuration

The Event trigger has two settings. **Event type** is the Grail table it watches: `events`, `bizevents`, `security.events` or `dt.system.events`. **Filter query** is a DQL matcher evaluated on each event as it arrives. For business events, choose `bizevents` and put the business event's own type in the filter as `event.type`. The actor needs `storage:bizevents:read` (§1).

The shape in an exported workflow (illustrative values):

```
trigger:
  eventTrigger:
    isActive: true
    triggerConfiguration:
      type: event
      value:
        eventType: bizevents
        query: event.type == "com.company.order-completed" and total &gt; 1000
```

Sending Business Events

Send **flat** JSON. *"Every top-level attribute is stored as a top-level field in Grail; nested JSON objects are stored as strings."* A field inside a nested `data` object therefore cannot be filtered on or read on its own. In the pure-JSON format, set `event.type` and `event.provider` as top-level fields, as Dynatrace's example does. (In the CloudEvents format, `type` maps to `event.type` instead.)

```
curl -X POST "https://&lt;environment-id&gt;.live.dynatrace.com/api/v2/bizevents/ingest"
\
  -H "Authorization: Bearer &lt;platform-token&gt;" \
  -H "Content-Type: application/json" \
  -d '{
    "event.type": "com.company.order-completed",
    "event.provider": "com.company.shop",
    "order_id": "ORD-12345",
    "customer": "ACME Corp",
    "total": 1500
  }'
```

The platform token needs the `openpipeline:bizevents:ingest` scope. The same page documents two alternatives: an OAuth bearer token on `https://<environment-id>.apps.dynatrace.com/platform/classic/environment-api/v2/bizevents/ingest` (*"This URL only supports OAuth authentication."*), and a Classic access token with the `bizevents.ingest` scope, sent as `Authorization: Api-Token …` to the `live` URL above.

Event Data

Top-level fields are read directly in tasks:

```
{{ event()["order_id"] }}
{{ event()["customer"] }}
{{ event()["total"] }}
{{ event()["event.type"] }}
```

Before you deploy, run **Query past events** in the trigger, or `fetch bizevents, from:-1h | filter event.type == "com.company.order-completed"`, to confirm the events arrive with the fields your filter uses.

> **Sources:** Ingest business events via API (DT docs) — *"type : Automatically converted to event.type"* (CloudEvents), the pure-JSON example, and the platform-token scope; Event triggers for workflows (DT docs) — event types and permissions.

Business events never open problems

That is a feature of this trigger, not a gap. A record ingested through `/api/v2/bizevents/ingest` lands in Grail as a business event: queryable, and able to trigger a workflow exactly as shown above — but it never raises a Davis problem and never enters problem correlation. `dt.smartscape_source.id`, the correlation key that decides which Davis events merge into one problem (AIOPS-03 §1), is not part of this path and does not apply here.

If what you actually want is a problem, this is the wrong API. Custom *Davis* events go to `/api/v2/events/ingest` with an `entitySelector` naming the entity the signal is about. Omitting that selector is the classic mistake: the event associates with the environment entity rather than a real one — so instead of failing to merge, it can merge with other alerts on that same entity that arrive within the correlation timeframe, into one problem that names nothing actionable. S2D-05 works through that failure in a scheduled-workflow context.

> **Sources:** Ingest an event — POST /api/v2/events/ingest (DT docs) — "If not set, the event is associated with the environment (`dt.entity.environment`) entity.", Avoid overalerting (DT docs) — *"Any two alerts that reference the same Smartscape entity and arrive within the established timeframe are merged into a single problem."*

7. Trigger Data and Expressions

Accessing Trigger Data

Expression	Returns	Example
`{{ event() }}`	Full event object	`{"event.name": "...", ...}`
`{{ event()["field"] }}`	Specific field	`"High response time"`
`{{ event().get("field") or "default" }}`	Field with a fallback (also covers a field that is present but null)	`"n/a"`
`{{ execution() }}`	Current execution (id, state, started_at, workflow)	`{"state": "RUNNING", ...}`

Expression	Returns	Example
`{{ input("name") }}`	A workflow input (on-demand / API runs)	`"prod"`
`{{ problem_link() }}`	URL of the problem (Problem trigger only)	`https://…/ui/apps/dynatrace.davis.problems/problem/…`

There is no `trigger()` expression. Field names contain dots, so use bracket access: `event()["event.name"]`, not `event().event.name`.

Common Event Fields by Trigger Type

Detected Problem (the `dt.davis.problems` record):

```
{{ event()["display_id"] }}           # P-260911127
{{ event()["event.id"] }}             # Problem ID ({PID} in Classic)
{{ event()["event.name"] }}           # Problem title
{{ event()["event.category"] }}       # AVAILABILITY, ERROR, SLOWDOWN,
RESOURCE_CONTENTION, CUSTOM_ALERT, ...
{{ event()["event.status"] }}         # ACTIVE or CLOSED
{{ event()["event.severity"] }}       # 1-5 (1 = most severe); stable, may be absent
{{ event()["event.start"] }}          # When the problem opened
{{ event()["affected_entity_ids"] }}  # Array of entity IDs
{{ (event().get("root_cause.smartscape_entity") or {}).get("name", "n/a") }}  # Root
cause; often absent or null
{{ event()["management_zones"] }}     # Legacy — not present on problem records (0 of
15,878 over 30 days); do not route on this
{{ problem_link() }}                  # Link to problem
```

Davis Event: the `dt.davis.events` record — `event.name`, `event.category`, `event.status`, `event.start`, `dt.source_entity` (see §3).

Schedule: no event payload. Use `{{ execution().started_at }}` or `{{ now() }}`.

Classic placeholder and old-template mapping

Templates written for Dynatrace Classic, and earlier revisions of this series, read fields the problem record does not have. Depending on how a missing field is read, the task fails with *Undefined variables* (bracket access) or the value is empty and every comparison is false (`.get()` with a default) — the second is silent. Use this mapping:

Replace	With	Note
`event()["title"]`	`event()["event.name"]`	`{ProblemTitle}`
`event()["status"]`	`event()["event.status"]`	Values `ACTIVE` / `CLOSED`, never `OPEN`
`event()["start_time"]`, `event()["end_time"]`	`event()["event.start"]`, `event()["event.end"]`	`{startTime}`, `{endTime}`
`event()["problem_url"]`	`{{ problem_link() }}`	`{ProblemURL}`; Problem trigger only
`event()["problem_id"]`	`event()["event.id"]`	`{PID}`
`event()["severity"] == "CRITICAL"`	`event()["event.category"] == "AVAILABILITY"` (kind of problem), or `(event().get("event.severity") | int(5)) <= 2` (1–5 scale)	`event.severity` is `stable` (semantic dictionary, read 11/11/2026); see WFLOW-04 §3

> **Sources:** Jinja expressions for Workflows (DT docs) — the expression list (`event()`, `execution()`, `input()`, `problem_link()`). Upgrade guide — alerting and notifications (DT docs) — the placeholder-to-field mapping; `{ProblemURL}` → `{{ problem_link() }}`: *"Evaluates only in workflows with a Davis problem trigger"*. **Dictionary:** `event.name`, `event.status`, `event.category` (`stable`), `event.severity` (`experimental`), read 09/24/2026.

8. Davis Problem Event Payload Reference

> **Rewritten 09/24/2026 from `fetch dt.davis.problems` on a live tenant and the upgrade guide's placeholder mapping.** The previous revision of this table listed `title`, `problem_id`, `status`, `start_time`, `impacted_entities` and `problem_url` as always present. None of them is on the problem record. Payload fields can change between releases: re-run `fetch dt.davis.problems, from:-24h | limit 1` before building routing on an uncommon field.

Workflows on a Detected Problem trigger receive one `event` object: the `dt.davis.problems` record at the moment the trigger fired. Sections 2 and 7 show common access patterns; this section is the reference: the fields you can rely on, when they appear, how they join to DQL, and which downstream notebooks consume them.

8.1. Top-Level Field Reference

Field	Type	Present	Notes
`event.kind`	string	Always	`"DAVIS_PROBLEM"` for this trigger. Distinguishes from `DAVIS_EVENT` (raw signals).
`display_id`	string	Always	Human-facing problem ID (`"P-260911127"`). Use in notifications and ticket subjects.
`event.id`	string	Always	Problem ID (`{PID}` in Classic). Use to **join to `fetch dt.davis.problems`** (see §8.3).
`event.name`	string	Always	Problem title (`{ProblemTitle}`).
`event.category`	string	Always	`AVAILABILITY` , `ERROR` , `SLOWDOWN` , `RESOURCE_CONTENTION` , `CUSTOM_ALERT` , `MONITORING_UNAVAILABLE` , `INFO` . `{ProblemSeverity}` maps here.
`event.status`	string	Always	`ACTIVE` or `CLOSED` . The documented open/close discriminator.
`event.status_transition`	string	Always	`CREATED` , `UPDATED` , `REFRESHED` , `RESOLVED` , `CLOSED` . `experimental` in the dictionary.
`event.severity`	long, 1–5	Usually	1 = most severe, 5 = least. `stable` (dictionary, read 11/11/2026). Typed `long` in Grail (validation tenant, 10/06/2026); keep `| int(5)` in templates, which also covers a missing value. No longer defaulted from SaaS 1.348 (WFLOW-04 §3).
`event.start`	timestamp	Always	When the problem opened (`{startTime}`).
`event.end`	timestamp	Closed only	When the problem closed (`{endTime}`).
`event.description`	string	Usually	Markdown problem details (`{ProblemDetailsText}`).

Field	Type	Present	Notes
`affected_entity_ids`	string[]	Always	Classic entity IDs. Deprecated in favor of `smartscape.affected_entities`.
`smartscape.affected_entities`	record[]	Usually	Each element has `id`, `type`, `name` — the entity *name* without an extra lookup.
`affected_entity_types`	string[]	Always	**Deprecated** in the semantic dictionary (read 11/11/2026). Entity types touched by this problem. Useful for routing without enumerating IDs.
`root_cause_entity_id`	string	When determined	Deprecated in favor of `root_cause.smartscape_entity`. Absent on most problems (102 of 540 carried it over 24 h on a validation tenant, 10/06/2026) — never bracket-read it.
`root_cause.smartscape_entity`	record	When determined	Root-cause node: `id`, `type`, `name`. The replacement field; absent just as often (127 of 540 over the same 24 h). Read with `.get()`.
`entity_tags`	string[]	When tagged	`key:value` strings (`"team:checkout"`), not objects. Deprecated in favor of `primary_tags.*`.
`primary_tags.*`	string	When enriched	Primary Grail tags propagated from the alerting events. The upgrade guide maps Classic `{Tags}` here. Values from every grouped event accumulate on the problem.
`dt.davis.impact_level`	string[]	Usually	`{ProblemImpact}`: every impact level observed across the problem's events (Application, Environment, Infrastructure, Services, Synthetic).
`dt.davis.event_ids`	string[]	Always	The Davis events grouped into this problem.
`maintenance.is_under_maintenance`	boolean	Always	Whether the problem falls in a maintenance window.

Field	Type	Present	Notes
`dt.duration_marker`	duration (ns)	Usually	Stepped open-duration marker used by the trigger's Minimum duration option.
`resolved_problem_duration`	duration (ns)	Closed only	Final duration once resolved.

There is no `problem_url` field. Use the `{{ problem_link() }}` expression.

> **Sources:** Event triggers for workflows (DT docs) — *"To browse past occurrences, explore available fields, and test filter conditions before configuring the trigger, run this query in a notebook"*. Upgrade guide — alerting and notifications (DT docs) — the placeholder mapping and the deprecations: *"root_cause_entity_id and root_cause_entity_name are deprecated in favor of root_cause.smartscape_entity, and affected_entity_ids in favor of smartscape.affected_entities."* **Dictionary:** `event.name`, `event.status`, `event.category` (`stable`), `event.severity`, `event.status_transition` (`experimental`), read 09/24/2026. **Observed 09/24/2026:** the *Present* column records a 7-day field-presence count on a validation tenant — what one tenant's payloads carried, not a documented guarantee.

Where the entity fields come from. `affected_entity_ids`, `root_cause_entity_id`, and `smartscape.affected_entities` are populated from the Davis events Dynatrace grouped into this problem, and that grouping runs on `dt.smartscape_source.id` — the Smartscape entity ID an event carries when it is properly attributed (84.9% of Davis events on a validation tenant over 7 days, 08/11/2026). Events naming the same entity within the correlation window merge into the single problem this payload describes.

That has a direct consequence for anyone reading these fields in a workflow: a problem assembled from **mis-attributed** events arrives with those fields populated by the *environment* entity rather than the thing that broke, so routing that branches on `affected_entity_types`, `entity_tags`, or `root_cause_entity_id` silently falls through to its default — the environment entity carries none of the team or service tags the routing expects. If a workflow keeps landing on the fallback branch, suspect the upstream detector's event template before rewriting the routing logic. Note the accompanying symptom is over-merge, not a problem-count spike: an event template that leaves `dt.smartscape_source.id` unset makes every such alert name the same entity, so they collapse into one implausibly broad problem. AIOPS-03 §1 covers the correlation rules, AIOPS-02 §8 finds unattributed detectors in your own tenant, and ALERT-99 §3 explains which Davis data object to count when you audit.

> **Sources:** Avoid overalerting (DT docs).

8.2. Lifecycle Field Presence

With **Problem state** set to *active or closed*, a single workflow fires when the problem opens and again when it closes (and on field changes, if **Updates** is enabled). The payload differs between the two, so guard logic that assumes every field is populated will mis-route close events.

Field	`event.status` = `ACTIVE`	`event.status` = `CLOSED`
`display_id`, `event.id`, `event.name`, `event.category`	✓	✓
`event.start`	✓	✓
`event.end`, `resolved_problem_duration`	—	✓
`root_cause_entity_id`	sometimes empty	populated when determined
`affected_entity_ids`, `smartscape.affected_entities`	initial set, may expand	final set

`event.status_transition` has five values (`CREATED`, `UPDATED`, `REFRESHED`, `RESOLVED`, `CLOSED`), and a close record can carry `RESOLVED` or `UPDATED` rather than `CLOSED`. Branch on `event.status`, which the upgrade guide uses for the close notification (*"build a second workflow filtering event.status == "CLOSED""*), then read only the fields present for that branch.

8.3. Joining the Payload to DQL

The workflow payload is a snapshot. For longer-window analysis (problem history, MTTR, fleet-wide patterns), join the payload's `event.id` to the canonical Grail table:

```
// Hydrate the workflow payload against the full Davis problems record.
// Substitute {{ event()["event.id"] }} via a DQL workflow task.
fetch dt.davis.problems, from:-7d
| filter event.id == "{{ event()['event.id'] }}"
| fields display_id,
         event.name,
         event.category,
         event.status,
         event.start,
         event.end,
         smartscape.affected_entities,
         affected_entity_ids,
         root_cause_entity_id
| limit 1
```

Why `dt.davis.problems` and not `dt.davis.events`: `dt.davis.problems` is the canonical problems data object; `dt.davis.events` carries only `DAVIS_EVENT` records (raw signals that *feed* problem

detection, not the problems themselves). Querying `dt.davis.events` for problem records returns zero rows on modern tenants.

For aggregate views (e.g., "all problems for the same entity this week") swap the filter:

```
fetch dt.davis.problems, from:-7d
| filter in("{{ event()['affected_entity_ids'][0] }}", affected_entity_ids)
| summarize problem_count = count(), by:{event.category, event.status}
| sort problem_count desc
```

8.4. Common Access Patterns

```
# Display ID and link — every notification
{{ event()["display_id"] }}
{{ problem_link() }}

# Category-based routing (see WFLOW-04 §3 for severity tiers)
{% if event()["event.category"] == "AVAILABILITY" %} page on-call
{% elif event()["event.category"] == "ERROR" %}        notify channel
{% else %}                                              log only
{% endif %}

# Lifecycle branch — distinguishes open from close in the same workflow
{% if event()["event.status"] == "CLOSED" %} close ticket
{% else %}                                   create/update ticket
{% endif %}

# Team routing on a primary tag (preferred; set primary_tags.team in your environment
first).
# Values accumulate across the problem's events, so this can hold more than one team.
{{ event().get("primary_tags.team") or "unassigned" }}

# Custom-tag access on the deprecated entity_tags (an array of "key:value" strings)
{% for tag in event().get("entity_tags", []) %}
  {% if tag.startswith("team:") %}{{ tag.split(":", 1)[1] }}{% endif %}
{% endfor %}

# Entity-type filtering — route by what kind of entity is involved
{% if "SERVICE" in event()["affected_entity_types"] %} service-team channel
{% elif "HOST" in event()["affected_entity_types"] %}   infra channel
{% endif %}

# Affected entity names (no extra lookup — smartscape.affected_entities carries names)
{% for ent in event().get("smartscape.affected_entities", []) %}
  {{ ent["name"] }} ({{ ent["type"] }})
{% endfor %}
```

8.5. Cross-References

- **WFLOW-04 §3 *Routing by Severity*** consumes `event.severity` and `event.category` from this payload to fan notifications out to team channels.
- **WFLOW-04 §4 *Routing by Team/Service*** reads entity tags (custom tags like `team:checkout`) and `affected_entity_types`.
- **WFLOW-05 *Incident Management*** uses `display_id`, `{{ problem_link() }}`, `event.status`, and `event.start` to create and update tickets through the lifecycle.
- **WFLOW-07 *Remediation*** branches on `root_cause_entity_id` and `event.name` to pick the right runbook task, and joins `event.id` to `dt.davis.problems` for richer context before acting.

8.6. Caveats

- **Old field names fail, sometimes silently.** A template that reads `title`, `status`, `problem_url` or `start_time` either errors (*Undefined variables*) or, when read through `.get()`, prints nothing and compares false. If a task errors on a template, or a notification arrives with blank fields, check the field names against §7's mapping first.
- **Re-verify before load-bearing logic.** This reference is dated 09/24/2026. The payload is product surface and can change in a sprint. Spot-check a real payload (a logging task, or `fetch dt.davis.problems, from:-24h | limit 1`) in your tenant before relying on a field in production routing.

Query Detected Problems

```
// Recent detected problems that could trigger workflows — one row per problem.
// dt.davis.problems holds one record per problem (540 records = 540 distinct event.id
over 24 h on a
// validation tenant, 10/06/2026). `fetch events | filter event.kind == "DAVIS_PROBLEM"`
holds one record
// per problem *update*: its latest 20 rows covered only 2 problems.
fetch dt.davis.problems, from:-24h
| fields event.start,
         display_id,
         event.name,
         event.category,
         event.status,
         smartscape.affected_entities,
         root_cause.smartscape_entity
| sort event.start desc
| limit 20
```

```
// Distinct problems by category that were ACTIVE at any point in the last 7 days
// Counted with countDistinctExact(display_id) since 09/24/2026: `events` holds one
record per
// problem update, so count() over-reported problems 4-35x on the validation tenant.
// Field names corrected 08/12/2026: on `events`, a Davis problem carries `event.status`
and
// `event.category` — there are no bare `status` / `severity` fields, so those filters
matched
// nothing. `event.status` values are ACTIVE / CLOSED — "OPEN" is not one of them.
fetch events, from:-7d
| filter event.kind == "DAVIS_PROBLEM"
| filter event.status == "ACTIVE"
| summarize problem_count = countDistinctExact(display_id), by:{event.category}
| sort problem_count desc
```

```
// Business events that could trigger workflows
fetch bizevents, from: now() - 24h
| summarize event_count = count(), by:{event.type}
| sort event_count desc
| limit 20
```

Next Steps

Now that you understand triggers, learn to send notifications:

Recommended Path

1. **WFLOW-03: Alert Notification Basics** - Slack, Teams, email notifications
2. **WFLOW-04: Advanced Notification Routing** - Conditional routing
3. **WFLOW-07: Problem-Triggered Remediation** - Auto-remediation patterns

Key Takeaways

- **Detected Problem** triggers for AI-detected incidents
- **Davis Event** triggers for per-alert reactions to anomaly detectors (thresholds live in the detector)
- **Schedule** triggers for recurring tasks
- **On-Demand** triggers for testing and API integration
- **Event** triggers for business events
- Use expressions like `{{ event()["field"] }}` to access data

Summary

In this notebook, you learned:

- All five trigger types and when to use each
- How to configure Detected Problem triggers with filters
- When to use a Davis event trigger instead of a Problem trigger
- Cron expressions for schedule triggers
- On-demand execution via UI and API
- Business event triggers for custom automation
- How to access trigger data in expressions

References

- Workflow triggers (DT docs)
- Workflow reference / Jinja expressions (DT docs)
- Davis Problems app (DT docs)
- Business Observability umbrella (DT docs)
- Workflows umbrella (DT docs)
- Cron expression sandbox (crontab.guru)
- Upgrade guide — alerting and notifications (DT docs)
- Event triggers for workflows (DT docs)
- Workflow schedule trigger (DT docs)
- Ingest business events via API (DT docs)
- Automation client SDK (Dynatrace Developer)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.